

Beyond Random Splits: Evaluating Temporal Generalization

in Explainable CI/CD Failure Prediction — A Multi-Repository Study

Technical Report

Prepared by: Amgad Magdy Ahmed

Abstract

CI/CD pipelines are central to modern software delivery, and predicting build failure before it occurs could help teams intervene early. This study builds an explainable failure-prediction pipeline using real GitHub Actions workflow-run data, first from a large open-source repository (pandas-dev/pandas, Phase 1), then extended to two additional, structurally different repositories (scikit-learn/scikit-learn and Facebook/react, Phase 2) to test generality. On pandas, an initial Random Forest model achieved a promising F1-score of 0.65 under random stratified evaluation; a feature-level data-leakage audit identified an execution-dependent duration feature and a preprocessing step fit on the full dataset, whose correction reduced F1 to 0.481 ± 0.058 under 5-fold cross-validation. Critically, under temporally realistic evaluation (training on older runs, testing on newer ones), performance dropped to $F1 = 0.15$ on a single chronological split and $F1 = 0.198$ under pooled expanding-window walk-forward evaluation. Phase 2 replicated this gap on both additional repositories: random-CV F1 of 0.516 ± 0.218 (scikit-learn, based on a small sample) and 0.661 ± 0.114 (react) fell to temporally ordered F1 scores of 0.33 (scikit-learn, single chronological split) and 0.26 (react, pooled walk-forward) respectively — the same direction of degradation across all three repositories, with gap magnitudes ranging from 0.19 to 0.40 F1 points. A follow-up exploratory experiment testing whether past-only historical failure rates could close this gap did not improve temporal performance in any repository: performance worsened in pandas and scikit-learn, while react worsened under pooled walk-forward evaluation and was unchanged on its single chronological split. A naive majority-class baseline achieved $F1 = 0.0$ on the failure class in both pandas and react, showing that the models retain non-trivial predictive signal beyond this trivial baseline, although that signal is weak. SHAP analysis identified workflow type as the dominant contributor, followed by time-of-day and day-of-week. The central finding of this study is methodological and was replicated across three structurally different repositories: apparent predictive performance under random validation substantially overstates real-world, future-oriented performance, revealing a temporal generalization problem that is arguably more important than the raw accuracy of any single model.

1. Introduction

Modern cloud-native applications increasingly rely on Kubernetes, Docker, microservices, and CI/CD pipelines to enable frequent, automated software deployments. While this automation improves delivery speed, it also introduces operational risk: configuration errors, resource contention, dependency conflicts, and outright pipeline failures. In many DevOps environments, these problems are detected only after a build has already run, wasting developer time and delaying releases.

There is a natural appeal to predicting failure before it happens, using data already available in the CI/CD system. This study investigates that idea directly — not by assuming it works, but by testing it rigorously enough to find out whether, and under what conditions, it actually does.

2. Research Questions

- RQ1: To what extent can a machine-learning model predict CI/CD build outcomes (success vs. failure) using workflow-level, pre-execution features, and does this performance hold under realistic (temporally ordered) evaluation?
- RQ2: Which workflow and timing-related factors contribute most strongly to predicted failure, as measured by SHAP feature attribution?
- RQ3: Can SHAP-based explanations provide interpretable, factor-level insight into the factors driving model predictions, and what limitations arise when interpreting these explanations under weak temporal generalization?

3. Dataset and Data Collection

Data was collected directly from the GitHub Actions REST API for the pandas-dev/pandas repository, a large, actively maintained open-source project. Up to 4,000 workflow runs were requested across 40 paginated requests, of which 3,998 fell into the four conclusion categories shown below (a small number of runs with other statuses, e.g. in-progress or startup failures, were not categorized); the data spans roughly nine days of activity (2026-08-10 to 2026-08-19).

Conclusion	Count
success	1797
skipped	1569
cancelled	392
failure	240

Runs labeled 'skipped' or 'cancelled' do not represent an executed pipeline's actual outcome and were excluded, leaving a cleaned dataset of 2,037 runs (1,797 success, 240 failure — an approximate 7.5:1 imbalance). This initial pull is referred to below as Pull A.

Data provenance note: due to Colab session interruptions during analysis, the workflow-run data had to be re-fetched from the GitHub Actions API twice mid-study, producing three slightly different snapshots referred to as Pull A, B, and C throughout this report (see Section 14.1 for full detail). Section 5 (initial model) and Section 6.2 (leakage-free cross-validation) use Pull A (2,037 cleaned runs); Section 6.1 (leakage-free single split) and Section 7 (temporal generalization) use Pull B (2,007 cleaned runs); Section 9 (historical feature experiment) uses a further refreshed Pull C. Results are internally consistent within each pull; absolute numbers compared across sections should be read as approximate agreement, not an exact controlled comparison. The core leakage correction ($F1\ 0.65 \rightarrow 0.481 \pm 0.058$) is entirely within Pull A and is unaffected by this issue.

4. Feature Engineering

- name_grouped — the workflow name, with any workflow occurring fewer than 20 times (within the relevant training data) merged into another category.

- `hour_of_day` and `day_of_week` — extracted from the run's creation timestamp.
- `duration_seconds` — initially included (elapsed time between run creation and completion), later removed after the leakage audit (Section 6).
- `historical_failure_rate` — an exploratory addition (Section 9, later replicated across repositories in Section 12): a strictly past-only failure rate per workflow type, computed only from runs preceding the current one.

Categorical features were one-hot encoded. Given the approximate 7.5:1 class imbalance, `class_weight='balanced'` was used during training rather than synthetic oversampling (e.g. SMOTE), and evaluation relied on precision, recall, and F1 for the failure class rather than raw accuracy.

5. Initial Model and Results

A Random Forest classifier (`n_estimators=200`, `class_weight='balanced'`, `random_state=42`) was trained using `duration_seconds`, `name_grouped`, `hour_of_day`, and `day_of_week`, evaluated with an 80/20 stratified random split.

Class	Precision	Recall	F1
success	0.95	0.97	0.96
failure	0.74	0.58	0.65

This result (F1 = 0.65, accuracy 0.93) was strong enough to warrant scrutiny before being trusted — which motivated the leakage audit below.

6. Leakage Audit

A feature-by-feature audit examined, for each feature, whether it was genuinely available before pipeline execution and whether its computation used any information from outside the training data.

Feature	Available before execution?	Leakage	Decision
<code>duration_seconds</code>	No — computed from the run's own start/end time	Confirmed (execution-dependent)	Removed
<code>name_grouped</code>	Grouping threshold was computed on the full dataset before the train/test split	Confirmed (preprocessing leakage)	Refit on training data only
<code>hour_of_day</code>	Yes — derived from run creation timestamp	None	Kept
<code>day_of_week</code>	Yes — derived from run creation timestamp	None	Kept

6.1 Leakage-Free Retrain

With `duration_seconds` removed and `name_grouped` refit on training data only, using the same model and split protocol (on Pull B, 2,007 cleaned runs — see Section 3 data provenance note; `train=1,605`, `test=402`):

Class	Precision	Recall	F1
success	0.95	0.89	0.92
failure	0.46	0.69	0.55

6.2 Leakage-Free Cross-Validation

5-fold stratified cross-validation, with `name_grouped` recomputed independently within each fold's training portion, gave a more reliable estimate:

Precision 0.439 ± 0.069 | Recall 0.536 ± 0.056 | F1 0.481 ± 0.058

The single-split F1 (0.65) was substantially optimistic; the cross-validated leakage-free F1 (0.481 ± 0.058) is a materially lower and more trustworthy estimate. The contribution of each individual leakage source to this reduction was not isolated experimentally and should not be attributed a specific percentage.

7. Temporal Generalization

Random stratified splitting assumes the underlying process is stationary. CI/CD failure behavior may not be: code changes, infrastructure conditions, and developer activity all evolve over time. This section tests whether the model generalizes to future, previously unseen time periods — a more realistic deployment scenario than random splitting.

7.1 Single Chronological Split

The dataset was sorted by creation time; the oldest 80% of runs were used for training and the newest 20% for testing, with no shuffling and the same leakage-free pipeline.

	Period	Size	Failure rate
Train (older)	2026-08-10 → 2026-08-18	1605	14.1%
Test (newer)	2026-08-18 → 2026-08-19	402	3.7% (15 cases)

Class	Precision	Recall	F1
success	0.97	0.91	0.94
failure	0.10	0.27	0.15

Failure F1 fell from 0.481 (random CV) to 0.15 under this future-oriented split — a substantial drop. With only 15 failure cases in this test window, however, the estimate alone carries high variance.

7.2 Expanding-Window Walk-Forward Evaluation

To obtain a more reliable estimate, the dataset was split into 5 chronological chunks and evaluated with an expanding-window walk-forward scheme (train on chunks 1..k, test on chunk k+1), pooling predictions across all four test windows (198 total failure cases).

Window	Failure rate	F1
1	9.2%	0.271
2	23.9%	0.037

3	12.5%	0.361
4	3.7%	0.115
Pooled	—	0.198

The failure rate itself varies sharply across windows (9.2% → 23.9% → 12.5% → 3.7%), suggesting substantial temporal non-stationarity in the observed CI/CD failure process rather than random noise. The pooled F1 of 0.198, built from 198 failure cases, provides stronger evidence that the single-split finding was not solely an artifact of one small, unlucky test window: temporal generalization was substantially weaker overall and highly variable across time windows, rather than uniformly weak.

F1 alone does not establish practical utility, particularly given how much the failure rate itself shifts across windows (3.7%–23.9%); this is addressed further in Section 14.

8. Answer to RQ1

Within the evaluated single repository and observation period, using only pre-execution features (workflow type, hour, day), CI build failure prediction shows non-trivial performance under random cross-validation ($F1 = 0.481 \pm 0.058$, reasonably consistent across folds though with some variability due to the limited number of failure cases) but weak and highly variable temporal generalization under realistic future-oriented evaluation (pooled walk-forward $F1 = 0.198$). The gap between these two evaluation strategies is itself a central finding: random-split evaluation produced a substantially higher performance estimate than the temporally ordered evaluation observed in this study, and the choice of evaluation protocol materially changes the answer to “can this be predicted?” This gap should be read primarily as evidence about the risk of random-split evaluation on non-stationary data, since the two estimates also come from different data snapshots (Pull A vs. Pull B — see Section 3 and 14.1) and are not a single perfectly controlled comparison. This conclusion is scoped to the pandas repository and roughly nine-day observation period studied in Phase 1 (see Section 14; Phase 2, Section 11, extends this to two further repositories).

9. Exploratory Historical Failure Rate Experiment (pandas, Single Repository)

Motivated by the temporal generalization gap, a follow-up exploratory experiment tested whether adding a strictly past-only historical failure rate per workflow type — computed only from runs preceding the current one — could recover some of the lost temporal performance.

Data provenances note: this experiment used a third data pull (Pull C), fetched in a later session and therefore slightly different again from both Pull A and Pull B (test set size 406 vs. 402, 18 failure cases vs. 15). As a result, the baseline F1 reported below (0.12) is not directly numerically comparable to Section 7.1's F1 of 0.15 — the two numbers come from different snapshots of the same live API, not the same held-out data. The Model A vs. Model B comparison within this section is internally valid (both trained and evaluated on the same Pull C split), so the relative effect of the historical feature is a fair comparison; the absolute baseline number is not.

Evaluation	Baseline F1	+ Historical F1
Single chronological split	0.12	0.21
Expanding-window walk-forward (pooled)	0.198*	0.072

The single-split result was promising (F1 nearly doubled) but did not hold under expanding-window walk-forward evaluation, where the historical feature performed worse overall. A closer look revealed a history-depth-dependent pattern: historical F1 improved as more training history accumulated (window 4, with the deepest history, was the only window where the historical feature outperformed the baseline: 0.182 vs. 0.115), while in early windows with little accumulated history the feature added noise rather than signal.

** The pooled baseline F1 (0.198) shown here is the same figure reported in Section 7.2, reused as the reference point for this comparison rather than recomputed on a new snapshot; this numerical match should not be read as evidence that Pull B and Pull C produced identical predictions or test observations — the historical-feature column (0.072) was computed fresh within this experiment. See Section 3 and 12.1 for the full data-provenance picture.*

The historical failure-rate feature did not improve temporal generalization overall — its apparent benefit on the single chronological split did not persist under expanding-window walk-forward evaluation. A history-depth pattern was visible (window 4, with the deepest accumulated history, was the only window where the feature outperformed baseline: 0.182 vs. 0.115), but with only 4 windows this is exploratory: the results suggest a possible relationship between history depth and predictive utility, not evidence sufficient to establish that relationship. This experiment should be read as exploratory, not confirmatory, and reinforces the broader methodological finding of this study: single-split results are unreliable on this dataset and should not be trusted without a repeated evaluation across multiple windows.

10. Explainability (SHAP)

SHAP's Tree Explainer was applied to a Random Forest trained on the full leakage-free dataset, for interpretation purposes (not performance evaluation).

10.1 Pre-Audit Model (includes duration_seconds)

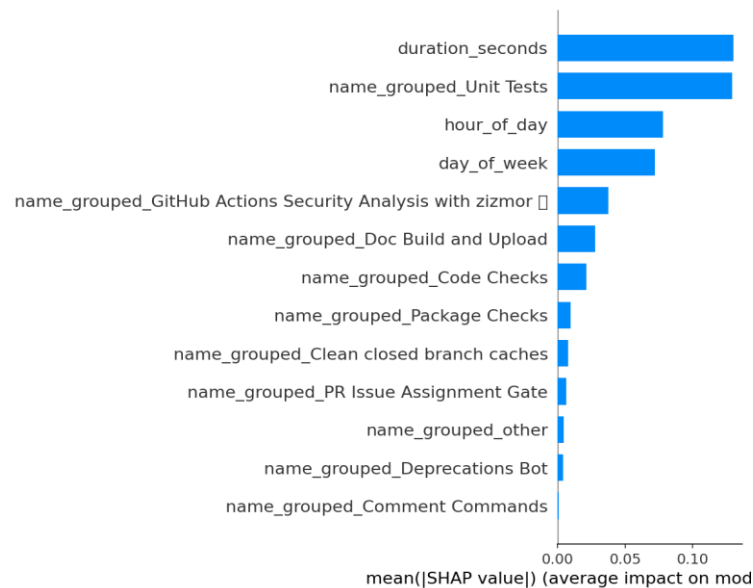


Figure 1. Global feature importance, pre-audit model.

In the pre-audit model, `duration_seconds` and `name_grouped_Unit Tests` were the two dominant contributors to model predictions. Since `duration_seconds` was later found to leak execution-time information (Section 6), this pre-audit analysis is retained only to illustrate how leakage manifested in the explainability results, not as a description of a usable predictive relationship.

10.2 Leakage-Free Model (`duration_seconds` removed)

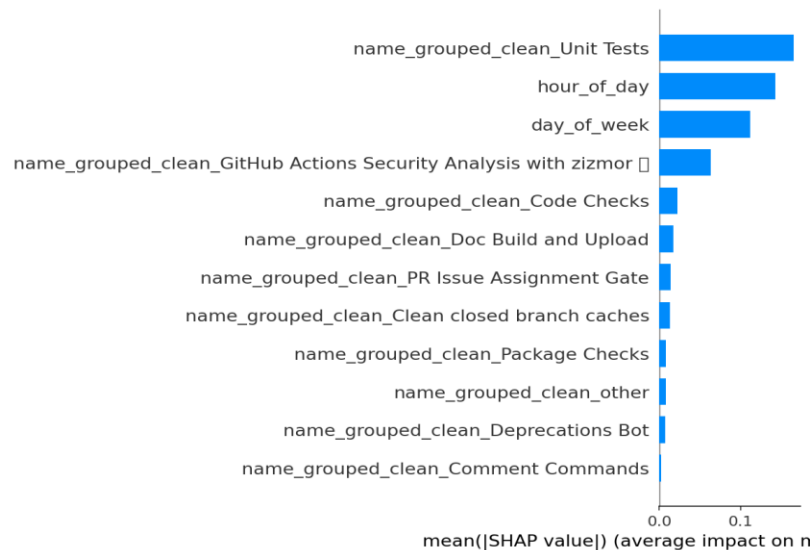


Figure 2. Global feature importance, leakage-free model.

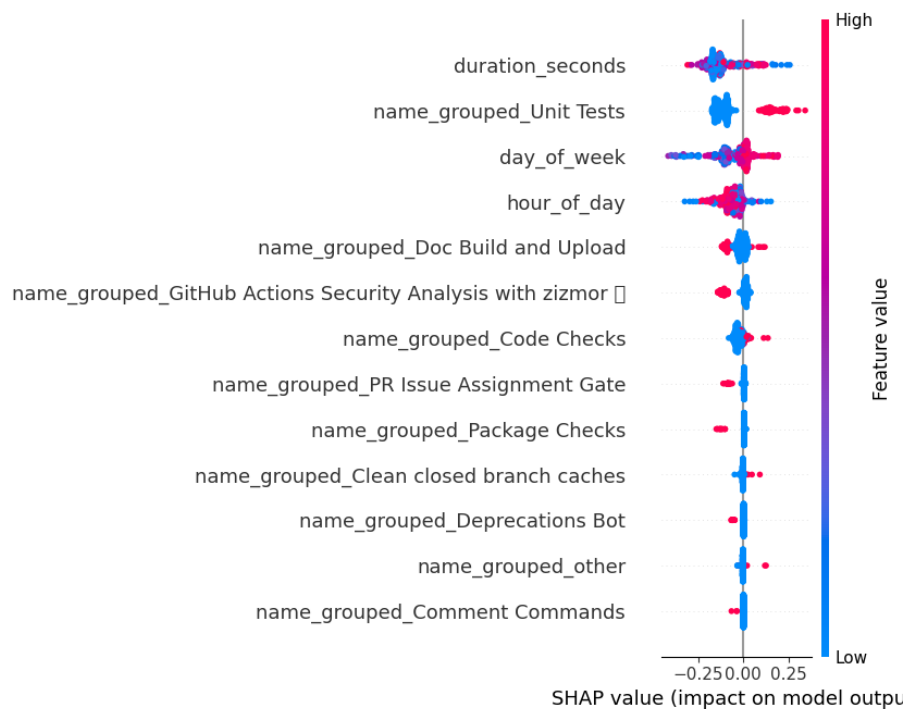


Figure 3. Direction of effect, leakage-free model (red = high feature value, blue = low).

With `duration_seconds` removed, `name_grouped_Unit Tests` remains the clear dominant driver, followed by `hour_of_day` and `day_of_week`. The Unit Tests indicator (a one-hot feature: 1 if the run belongs to the Unit Tests workflow, 0 otherwise) was among the strongest contributors to the leakage-free model's predictions in this dataset; the temporal features show a weaker, less linear pattern, consistent with the temporal non-stationarity documented in Section 7.

Given the confirmed temporal generalization gap, these SHAP explanations should be read as describing associations the model learned within this historical dataset, not as stable, causal, or reliably predictive relationships for future runs.

11. Phase 2: Multi-Repository Replication

Phase 1 (Sections 1–10) raised an open question in its limitations: whether the observed temporal generalization gap was specific to `pandas-dev/pandas` or a more general property of CI/CD failure data. Phase 2 addresses this directly by repeating the identical leakage-free pipeline — same core feature set (`name_grouped`, `hour_of_day`, `day_of_week`) and Random Forest configuration, with the temporal evaluation protocol adapted to dataset size and failure count — on two additional, structurally different repositories: `scikit-learn/scikit-learn` (similar domain and language to `pandas`, but a much smaller and shorter dataset) and `facebook/react` (a different language ecosystem — JavaScript — and a larger, more active project). No new features or model changes were introduced in this phase; only the source repository changed.

11.1 Datasets

Repository	Cleaned runs	Failures	Observation period
<code>pandas-dev/pandas</code> (Phase 1)	2,007–2,037*	240	~9 days
<code>scikit-learn/scikit-learn</code>	857	25	~3 days
<code>facebook/react</code>	1,354	59	~13 days

** `pandas` figures vary slightly by data pull; see Section 3 and 14.1. An `expressjs/express` pull was also attempted but yielded only 4 failures in 77 runs — too small to evaluate reliably — and was replaced with `facebook/react`.*

11.2 Random Cross-Validation vs. Temporal Evaluation, All Three Repositories

Repository	Random CV F1	Temporal F1	Gap
<code>pandas</code>	0.481 ± 0.058	0.198 (pooled, 4 windows)	-0.283
<code>scikit-learn</code>	0.516 ± 0.218*	0.33 (single split, 7 failures)	-0.186
<code>react</code>	0.661 ± 0.114	0.26 (pooled, 4 windows)	-0.401

** `scikit-learn`'s random-CV standard deviation (± 0.218) is nearly half the mean, reflecting only ~5 failure cases per fold; this estimate should be treated as directional, not precise, and `scikit-learn` was evaluated with a single temporal split rather than a full rolling-window due to its small sample.*

The direction of the gap — random-split evaluation producing a substantially higher F1 than temporally ordered evaluation — replicated in both additional repositories, with gap magnitudes (0.19–0.40 F1

points) of a broadly comparable order to pandas. This is the central Phase 2 finding: the temporal generalization gap identified in Phase 1 is not specific to pandas-dev/pandas.

12. Phase 2b: Historical Feature Replication and Naive Baseline

Phase 1’s exploratory historical-failure-rate experiment (Section 9) showed a mixed, single-repository result. Phase 2b repeats the same past-only historical failure rate feature (computed strictly from runs preceding the current one) on scikit-learn and react, using the same methodology as Section 9.

Repository	Baseline (temporal) F1	+ Historical F1	Effect
pandas	0.198 (pooled)	0.072 (pooled)	Worse
scikit-learn	0.33 (single split)	0.24 (single split)	Worse
react	0.26 (pooled) / 0.14 (single split)	0.185 (pooled) / 0.14 (single split)	Worse / No change

The historical failure-rate feature worsened temporal performance in all three repositories under the evaluation protocol used for each (pooled walk-forward for pandas and react; a single temporal split for scikit-learn, which had too few failures for a reliable walk-forward evaluation). In react specifically, the single-split result was unchanged (0.14 → 0.14) while pooled performance worsened (0.26 → 0.185). This is a consistent negative replication across repositories and evaluation designs, not a repository-specific anomaly — it strengthens rather than weakens the case that this specific feature does not address the temporal generalization gap at the data volumes available here, and suggests the underlying issue may be closer to genuine concept drift than to a simple lack of historical context (see Section 16, Future Work).

12.1 Naive Baseline Comparison

A remaining gap flagged in Phase 1’s limitations (Section 14.3) was the absence of a trivial baseline to contextualize the temporal F1 scores. A DummyClassifier predicting the majority class (“always success”) was evaluated on the same pooled walk-forward windows used for pandas and react.

Repository	Naive baseline F1 (failure class)	Model F1 (pooled temporal)
pandas	0.0	0.198
react	0.0	0.26

The naive baseline scores 0.0 on the failure class by construction (it never predicts failure). Both models achieve positive failure-class F1 beyond this trivial baseline, indicating that the models retain predictive signal under these temporal evaluations, although the signal is modest. This does not contradict the central finding — temporal generalization is still substantially weaker than random-split evaluation suggested — but it does address the concern that the low temporal F1 scores might be indistinguishable from a model with no signal at all.

13. Discussion

Summary of Main Findings

Finding	Result	Interpretation
---------	--------	----------------

Initial leaky model (pandas)	F1 = 0.65	Optimistic, later found to include leakage
Leakage-free random CV (pandas)	F1 = 0.481 ± 0.058	Moderate signal under random evaluation
pandas, temporal evaluation	F1 = 0.198 (pooled)	Substantial degradation
scikit-learn, temporal evaluation	F1 = 0.33 (single split)	Degradation replicated
react, temporal evaluation	F1 = 0.26 (pooled)	Degradation replicated
Historical failure-rate feature	Worse or unchanged, all 3 repos	Did not close the temporal gap
Naive majority-class baseline	F1 = 0.0 (pandas, react)	Models retain non-trivial signal
Dominant SHAP factor	Workflow type (Unit Tests)	Strongest learned association, not a causal claim

The overall trajectory of this study — F1 = 0.65 (leaky) → 0.481 ± 0.058 (leakage-free, random CV) → 0.15–0.20 (temporal) — illustrates a general risk in applied ML for software engineering: an evaluation protocol that does not account for data leakage or temporal structure can produce results that look strong but do not reflect real-world, future-oriented performance. The single most important methodological lesson from this study is that evaluation protocol choice can matter as much as, or more than, model or feature choice.

Phase 2 strengthens this interpretation by reproducing the same direction of temporal degradation in two additional repositories. Although the strength of the evidence differs across repositories — particularly because scikit-learn had relatively few failures and was evaluated with a single temporal split — the replication across pandas, scikit-learn, and react suggests that the observed temporal generalization problem is not solely a property of one repository. The negative results from the historical-failure-rate experiment further indicate that adding a simple cumulative historical feature is insufficient to address the observed gap.

The dominant predictive factor identified — workflow type (particularly Unit Tests) — is intuitive and consistent with raw descriptive statistics, lending some confidence to the explainability results even though the underlying predictive performance is limited.

The sharply varying failure rate across rolling windows (9.2% → 23.9% → 12.5% → 3.7% in pandas) is consistent with temporal distribution shift and possibly concept drift. However, the short observation periods used here (3–13 days per repository) do not constitute a rigorous concept-drift analysis — that would require dedicated drift-detection methodology and longer time series. This study establishes non-stationarity as a plausible explanation for the observed generalization gap, not a formally confirmed one.

14. Threats to Validity and Limitations

14.1 Dataset Limitations

- Three data pulls used: Colab session interruptions required re-fetching data twice mid-study (Section 3, Section 9). Pull A (2,037 cleaned runs) underlies Section 5 and 6.2; Pull B (2,007 cleaned

runs) underlies Section 6.1 and Section 7; Pull C (test set of 406 rows, 18 failures) underlies Section 9. Because the GitHub API returns the most recent runs at query time, each pull is a slightly different snapshot. Comparisons within a section (e.g. Model A vs. B in Section 9, or leaky vs. leakage-free within Pull A) are internally valid; absolute numbers compared across sections (e.g. Section 7.1's $F1=0.15$ vs. Section 9's baseline $F1=0.12$) are not a controlled comparison and should be read as approximate, not exact, agreement.

- Phase 1 was based on a single repository (pandas-dev/pandas), limiting the generalizability of its initial findings. Phase 2 (Section 11) partially addresses this by replicating the temporal generalization gap in scikit-learn/scikit-learn and facebook/react; however, all three repositories were observed over relatively short periods (3–13 days) and exclusively through GitHub Actions.
- Short observation period: data covers roughly nine days; longer-term temporal patterns (e.g. weekly or release-cycle effects) could not be assessed.
- Small and highly variable failure counts: 240 total failure cases, as few as 15 in a single temporal test window, and a failure rate that itself ranged from 3.7% to 23.9% across walk-forward evaluation windows — all of which limit the precision of reported metrics.

14.2 Feature Limitations

- Few features overall, and no commit-level information (code-change magnitude, files changed, lines added/deleted) or richer developer/project history context.
- No infrastructure-layer data: only Git/CI-level features were used; Kubernetes and operational signals were out of scope.

14.3 Model and Evaluation Limitations

- Single model family (Random Forest) with default hyperparameters; no systematic comparison against other model families or tuned configurations was performed. Reported results should not be interpreted as the maximum achievable performance for this feature set — performance may differ under systematic hyperparameter tuning or alternative model classes.
- F1 alone does not establish practical utility under sharply shifting class prevalence (Section 7.2). A naive majority-class baseline was added for the pooled temporal evaluations of pandas and react (Section 12.1, both scoring $F1 = 0.0$ on the failure class, indicating the models retain predictive signal beyond this trivial baseline), but was not evaluated across every repository and evaluation protocol — notably not for scikit-learn's single-split evaluation; broader baseline comparisons remain future work.
- Explanation usefulness not user-tested: SHAP outputs were evaluated for internal consistency, not for practical usefulness to DevOps practitioners.
- Historical-feature experiment (Section 9) used limited historical depth (at most ~1600 prior runs) and only 4 expanding-window walk-forward windows; its potential was not fully tested at larger scale.

14.4 Generalization

- Phase 2 (Section 11) partially addresses this: the temporal generalization gap replicated in two additional, structurally different repositories (scikit-learn, react), suggesting it is not specific to pandas-dev/pandas. However, all three repositories were sampled over short windows (3–13 days) via the same live API at a similar point in time, and none involved infrastructure beyond GitHub

Actions — broader claims about CI/CD systems generally (different CI providers, longer timeframes, closed-source/enterprise contexts) remain untested.

- scikit-learn’s replication is the weakest of the three (only 25 total failures, wide confidence interval on random CV); it should be treated as broadly consistent with, rather than strong independent confirmation of, the pattern seen in pandas and react.

15. Conclusion

This study set out to build an explainable CI/CD failure predictor, and ended up demonstrating something more useful: that apparently strong predictive performance for this task can look substantially stronger under random evaluation than it does under temporally realistic evaluation, particularly when the underlying process is non-stationary. On pandas (Phase 1), leakage correction reduced the failure-class F1 from 0.65 (leaky, single split) to 0.481 ± 0.058 (leakage-free, 5-fold cross-validation), and temporally realistic evaluation of the same leakage-free pipeline yielded $F1 = 0.15$ on a single chronological split and $F1 = 0.198$ under pooled expanding-window walk-forward evaluation.

Phase 2 tested whether this gap was specific to pandas by repeating the identical pipeline on two structurally different repositories. The gap replicated in both: scikit-learn fell from $F1 = 0.516 \pm 0.218$ (random CV) to 0.33 (temporal), and react fell from $F1 = 0.661 \pm 0.114$ to 0.26 (pooled temporal). A follow-up historical-failure-rate feature, intended to close this gap using strictly past-only information, did not improve temporal performance in any repository: performance worsened in pandas and scikit-learn, while in react it worsened under pooled walk-forward evaluation and was unchanged on the single chronological split. Thus, across all three repositories, the feature produced either a negative or null result under the evaluation protocol used for each repository. A naive majority-class baseline scored $F1 = 0.0$ on the failure class in both pandas and react, while the leakage-free models achieved positive F1, indicating predictive signal beyond the trivial baseline despite the temporal generalization gap.

Taken together, these findings indicate that temporal generalization — not model complexity alone — is a central and recurring challenge for pre-execution CI/CD failure prediction, and that this challenge is not confined to a single repository. The consistency of this pattern across three structurally different repositories is, in the authors’ view, the strongest contribution of this study — though, as noted in Section 14.4, all three were sampled over short windows via the same API, so broader claims about CI/CD systems in general remain to be tested.

16. Future Work

The central finding of this study is not simply that prediction is hard, but that prediction under distribution shift is the real challenge — which points future work toward temporal robustness rather than incremental accuracy gains:

- Longitudinal dataset: collect data over a substantially longer period. Phase 2b tested the historical-failure-rate feature on three repositories and found it consistently unhelpful at current data

volumes (a few hundred to ~2,000 runs per repository); it remains untested whether much deeper history (e.g. years, not days) would behave differently.

- Richer past-only features: rolling/recent failure rate over a fixed window, recent failure streaks, recent workload, and workflow-specific history beyond the single cumulative-average feature tested here — Phase 2b tested only one specific formulation of “historical information,” not the concept broadly.
- Change-aware features: commit- and PR-level information (code-change magnitude, files changed, lines added/deleted) that was out of scope for this phase.
- Concept drift detection and temporal adaptation: since the problem is prediction under a shifting failure-rate process rather than a static one, online learning or explicit drift-detection methods are a natural and arguably higher-value direction than adding more static features.
- Infrastructure-level signals (Kubernetes, CPU/memory, pod restarts) as originally envisioned in the broader research direction.
- Bayesian hyperparameter optimization, compared against the default configuration used here.
- A user study with practicing DevOps engineers to assess whether SHAP explanations are genuinely actionable, not just internally consistent.
- Risk-aware decision support (mapping predicted risk to deployment-strategy recommendations), once a temporally robust predictive signal is established.